RELATÓRIO TÉCNICO / 2026

VOA ÁGUIA



Integração contínua, testes automatizados, containerização e implantação em nuvem

JAVA 17 DOCKER GITHUB ACTIONS RENDER + ATLAS

INTEGRANTES

Mirella Ayumi Miyakawa
mirellaayumi@gmail.com

Gustavo Okamoto Ventura
gustavookamoto1945@gmail.com

Davi Simplicio Nunes Coelho
davisimplicio104@gmail.com

Guilherme Vinicius Gonçalves da Silva
guilhermevinicius22052007@gmail.com

TRILHA • NAVEGANDO PELO MUNDO DEVOPS

Desafio acadêmico | Aplicação Voa Águia - Sprint 2

13 de outubro de 2026

01 / APRESENTAÇÃO

Contexto e objetivos

Visão geral do sistema e dos requisitos técnicos atendidos no desafio acadêmico.

Sobre o projeto

O Voa Águia é uma API REST desenvolvida com Java e Spring Boot para apoiar a gestão de estratégias, ideias e projetos, com autenticação por JWT, perfis de acesso, acompanhamento de iniciativas, indicadores e documentação OpenAPI/Swagger. Seu modelo de organização de propostas pode ser aplicado a iniciativas de ESG, conforme a temática do desafio.

Objetivo da implementação DevOps

Aplicar automação ao ciclo de vida do backend: compilar, testar, construir uma imagem Docker e publicar o mesmo código validado em dois ambientes independentes (staging e produção acadêmica). O processo utiliza GitHub Actions, Render e MongoDB Atlas.

CI / Integração contínua

Maven + Java 17, testes com MongoDB temporário e build da imagem Docker.

CD / Entrega contínua

Deploy controlado por branch, confirmação de status LIVE e verificação HTTP de saúde.

Requisitos demonstrados

REQUISITO	EVIDÊNCIA / IMPLEMENTAÇÃO
Compilar e testar	Job Maven com MongoDB e relatórios de testes
Containerizar	Dockerfile multiestágio e imagem construída no CI
Orquestrar	Docker Compose local com API, rede e volume MongoDB
Publicar em dois ambientes	Render staging e produção, com bancos Atlas distintos
Documentar	README, screenshots e relatório com checklist

Delimitação ESG: o backend gerencia iniciativas e indicadores que podem ser usados em projetos ESG. As capturas apresentadas comprovam a infraestrutura e as rotas da API, não resultados quantitativos de sustentabilidade.

Arquitetura da solução

Separação explícita entre execução local, CI/CD e os ambientes em nuvem.



Ambiente local separado da nuvem

Docker Compose: executa api (Spring Boot) e mongo (MongoDB 7.0) na rede interna `voaaguia_network`, com volume persistente `mongo_data`. Na nuvem, o Render executa somente a aplicação, que acessa o MongoDB Atlas por conexão externa.

Observação de arquitetura: o banco H2 é usado em memória para auditoria e não mantém esses registros após reinício; o MongoDB Atlas mantém as entidades principais da aplicação.

Tecnologias: Java 17, Spring Boot 3.5.6, Maven, Spring Security/JWT, JUnit, Spring Data MongoDB, H2, Swagger/OpenAPI, Docker, Compose, GitHub Actions, Render e MongoDB Atlas.

Docker e execução local

Build multiestágio, serviços Docker Compose e comandos reprodutíveis.

Dockerfile da aplicação

```
FROM maven:3.9.9-eclipse-temurin-17 AS build
WORKDIR /workspace
COPY pom.xml .
RUN mvn -q dependency:go-offline
COPY src src
RUN mvn -q package -DskipTests

FROM eclipse-temurin:17-jre
WORKDIR /app
COPY --from=build /workspace/target/backend-2.0.0.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java","-jar","app.jar"]
```

A compilação acontece na imagem Maven. A imagem final usa apenas o JRE 17, reduzindo dependências de execução. O GitHub Actions valida e constrói a imagem; o Render, por sua vez, reconstrói a aplicação com o mesmo Dockerfile a partir do commit aprovado. Não há publicação da imagem do CI em um registry.

Orquestração local / docker-compose.yml

O Compose define dois serviços, ordem de inicialização com healthcheck do MongoDB, rede bridge dedicada, variáveis de ambiente e volume `mongo_data:/data/db` para persistência local.

Comandos executáveis

Subir containers	`docker compose up -d --build`
Listar serviços	`docker compose ps` / `docker ps`
Construir imagem isolada	`docker build -t voaaguia:teste .`
Parar sem excluir volume	`docker compose down`

Imagem observada no CI: tag `voaaguia:<commit SHA>`. Na execução local, o Docker Compose constrói a imagem do serviço `api`.

Pipeline CI/CD

Workflow: .github/workflows/ci-cd.yml | Script: scripts/render-deploy.sh

Eventos e política de publicação

EVENTO	VALIDAÇÃO CI	DESTINO CD
Pull request → develop/main	Build + testes + Docker build	Sem deploy
Push → develop	Build + testes + Docker build	Deploy staging
Push → main	Build + testes + Docker build	Deploy produção
Execução manual	Conforme branch escolhida	Conforme branch

Sequência automatizada

01 Checkout do repositório

02 Java 17 + cache Maven

03 mvn clean verify + MongoDB 7 no runner

04 docker build com o SHA do commit

05 Render API: publicar SHA validado

06 Aguardar LIVE e validar /healthz

Segurança e rastreabilidade

O token da API do Render é armazenado em `RENDER_API_KEY` (GitHub Secrets); os IDs e endereços ficam nas variáveis dos Environments; senhas e `MONGODB_URI` ficam no Render. A publicação informa o SHA do commit validado e confere a resposta do endpoint de saúde.

Execução local com Docker

Registros da aplicação e do banco de dados em execução no Windows com Docker Desktop.

Evidência 01 — Containers da aplicação e do banco

A saída de `docker ps` apresenta voaaguia-local-api-1 em execução na porta local 8080 e voaaguia-local-mongo-1 em estado healthy.

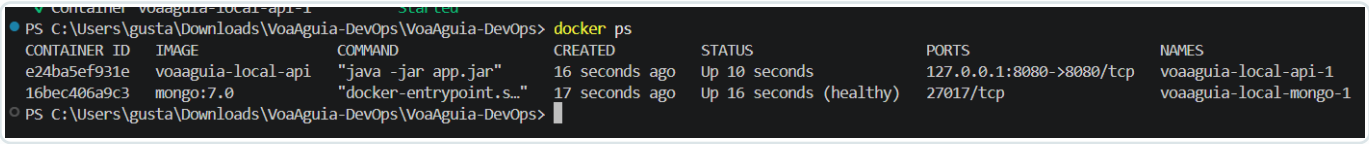


Figura 1. Execução dos containers Spring Boot e MongoDB no ambiente local.

Evidência 02 — Swagger em localhost

A interface Swagger UI carregou a documentação do backend VoaÁguia - Sprint 2 e suas rotas REST em `localhost:8080`.

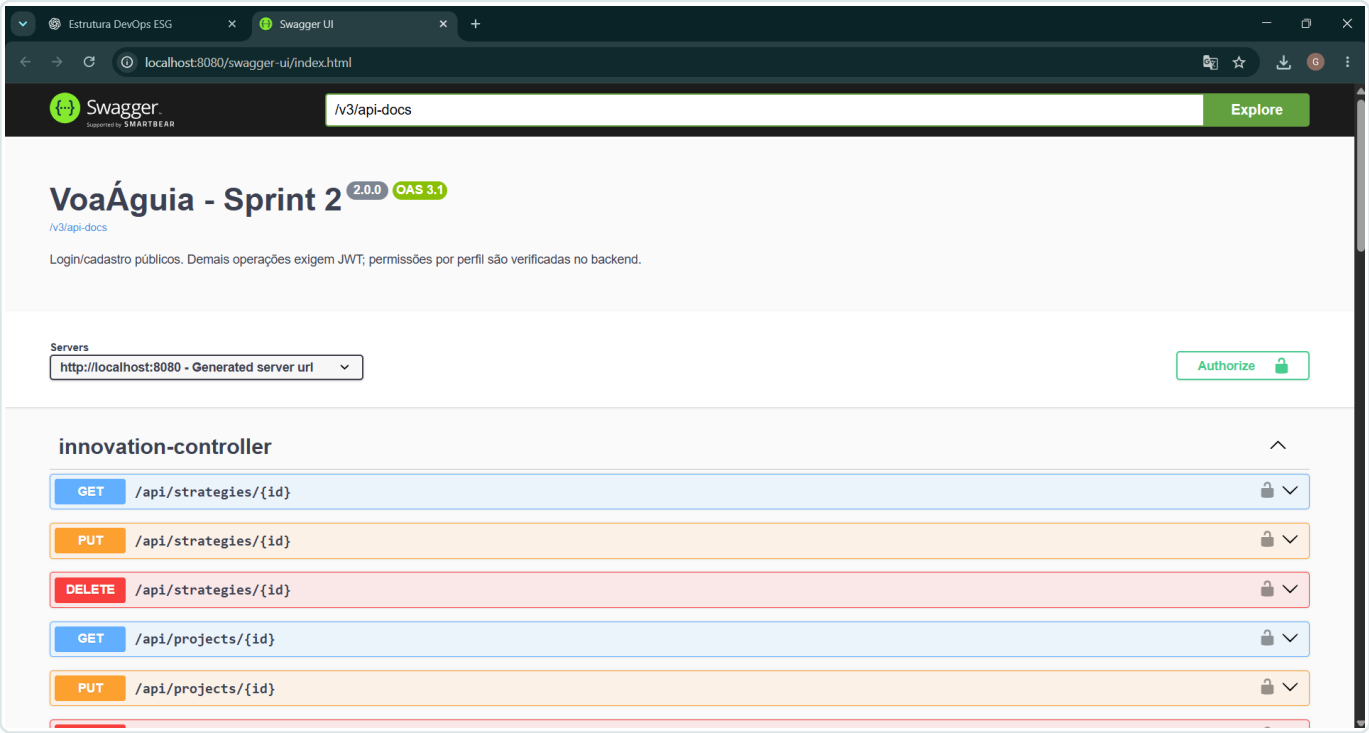


Figura 2. Swagger local em http://localhost:8080/swagger-ui/index.html.

Build, testes e imagem Docker

GitHub Actions aprovou as etapas de compilação Java, testes integrados e build da imagem.

Job: "Compilar, testar e construir imagem Docker" — resultado sucesso. O runner inicializou MongoDB, preparou Java 17, executou o Maven, construiu a imagem e publicou artefatos de relatórios de testes.

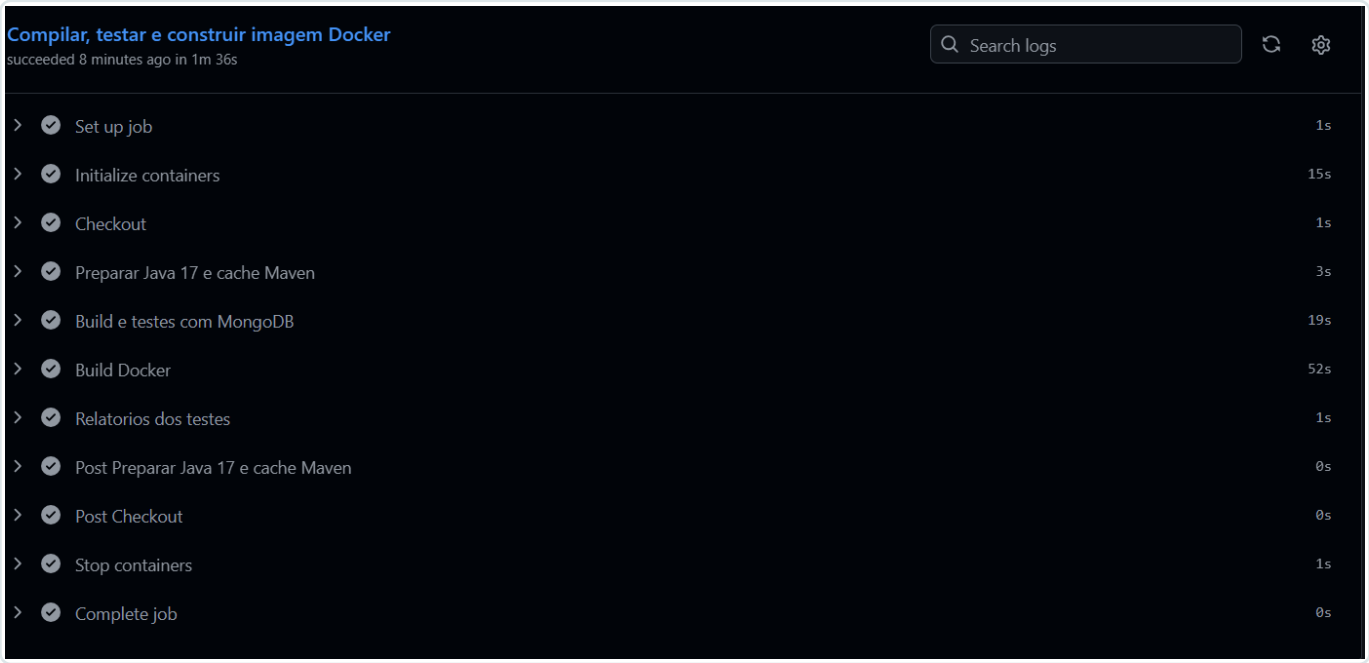


Figura 3. Etapas aprovadas: checkout, Java 17/cache Maven, build e testes com MongoDB, Docker build e relatórios.

Histórico geral do workflow

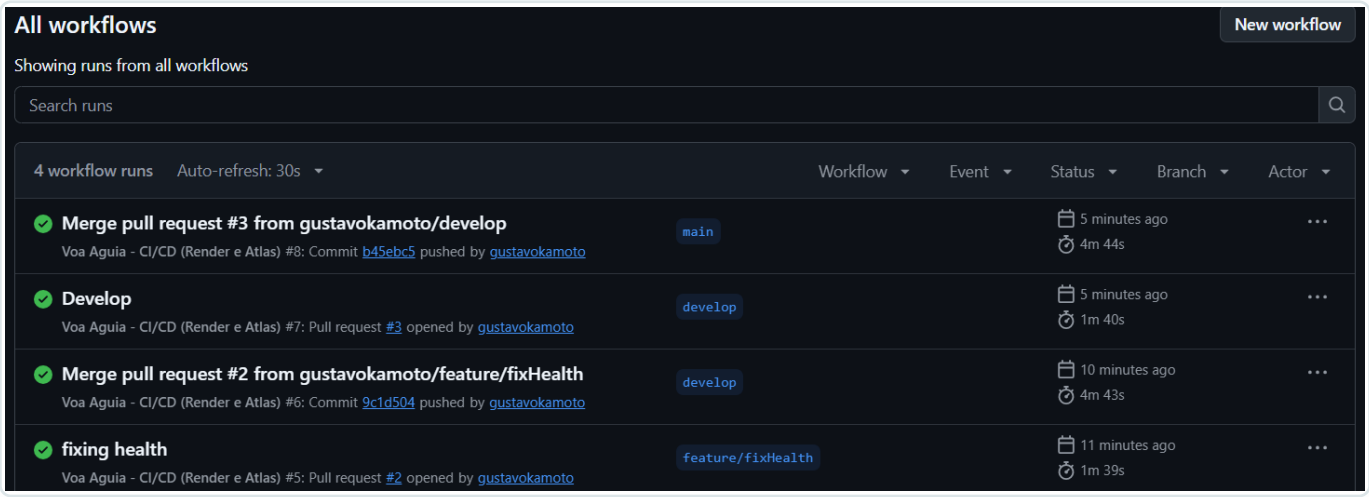


Figura 4. Quatro execuções recentes do workflow com indicador de sucesso, incluindo branches develop e main.

Escopo dos testes: o código possui testes de integração para autenticação, permissões, fluxos de ideias/projetos e disponibilidade do MongoDB, além de teste de auditoria. A captura demonstra aprovação do job; não informa, isoladamente, a contagem de testes executados.

Publicação em staging

Serviço de homologação no Render, com banco dedicado no MongoDB Atlas.

Branch: develop | Destino: Render Free | URL: https://esg-devops-staging.onrender.com

Evidência 03 — GitHub Actions → Render LIVE

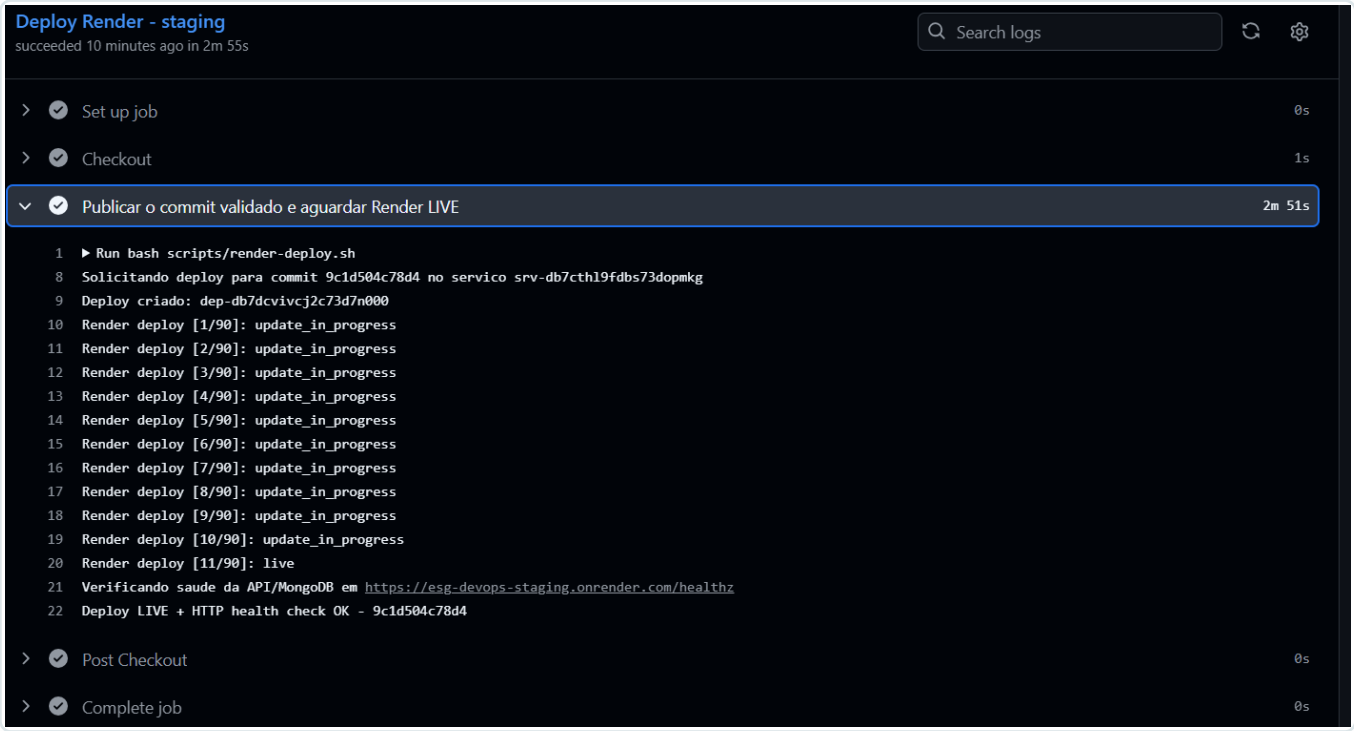


Figura 5. Job de staging concluído. Log confirma status LIVE e "Deploy LIVE + HTTP health check OK".

Evidência 04 — Swagger público de staging

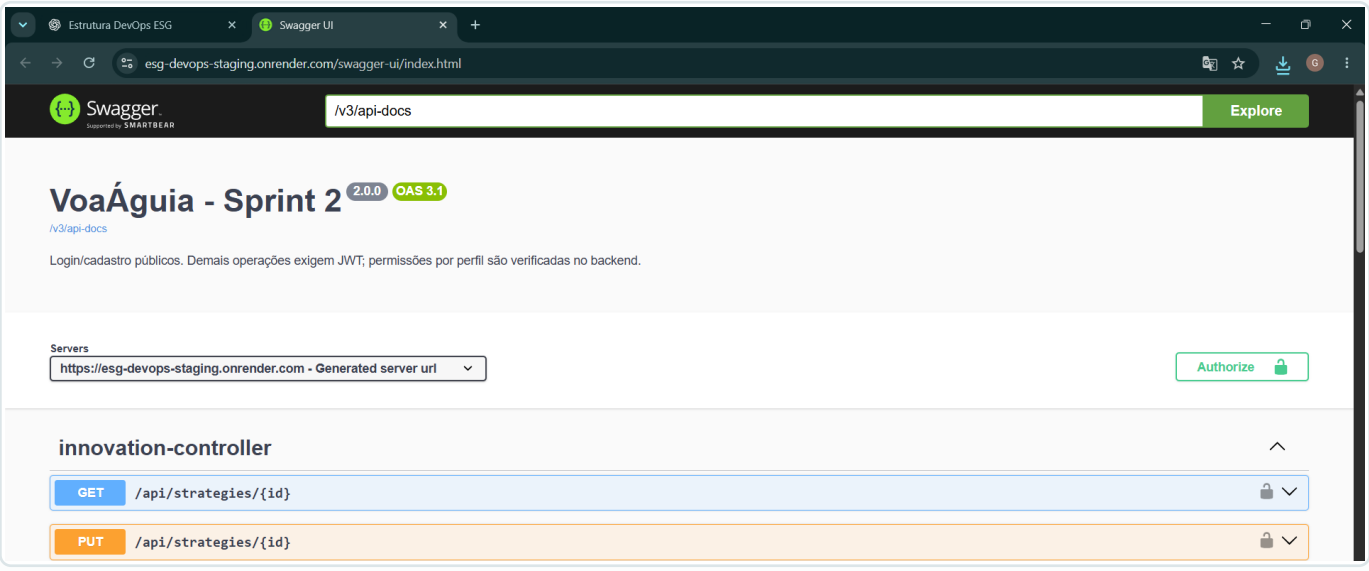


Figura 6. Swagger do Voa Águia acessível via HTTPS no ambiente de staging.

Publicação em produção

Ambiente acadêmico de produção, separado do staging por serviço e banco de dados.

Branch: main | Destino: Render Free | URL: https://esg-devops-production.onrender.com

Evidência 05 — GitHub Actions → Render LIVE

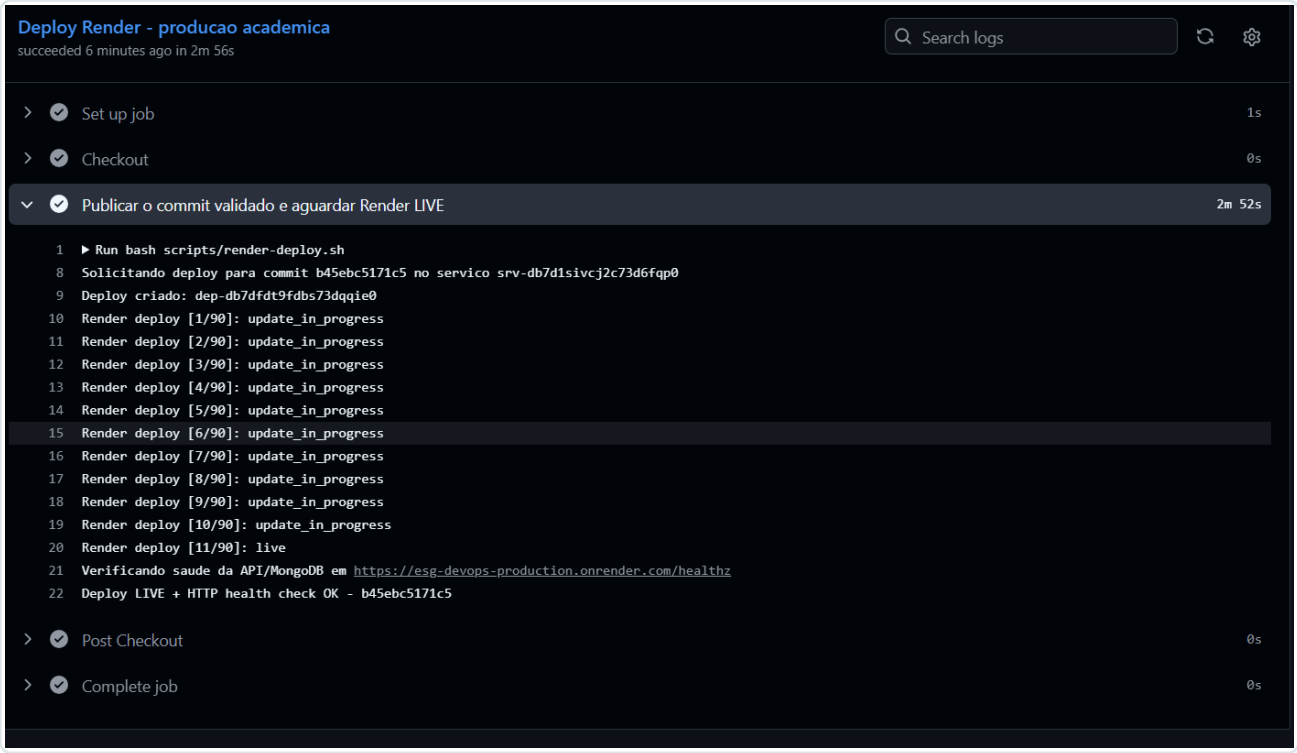


Figura 7. Job de produção concluído com sucesso; status LIVE e health check HTTP aprovados.

Evidência 06 — Swagger público de produção

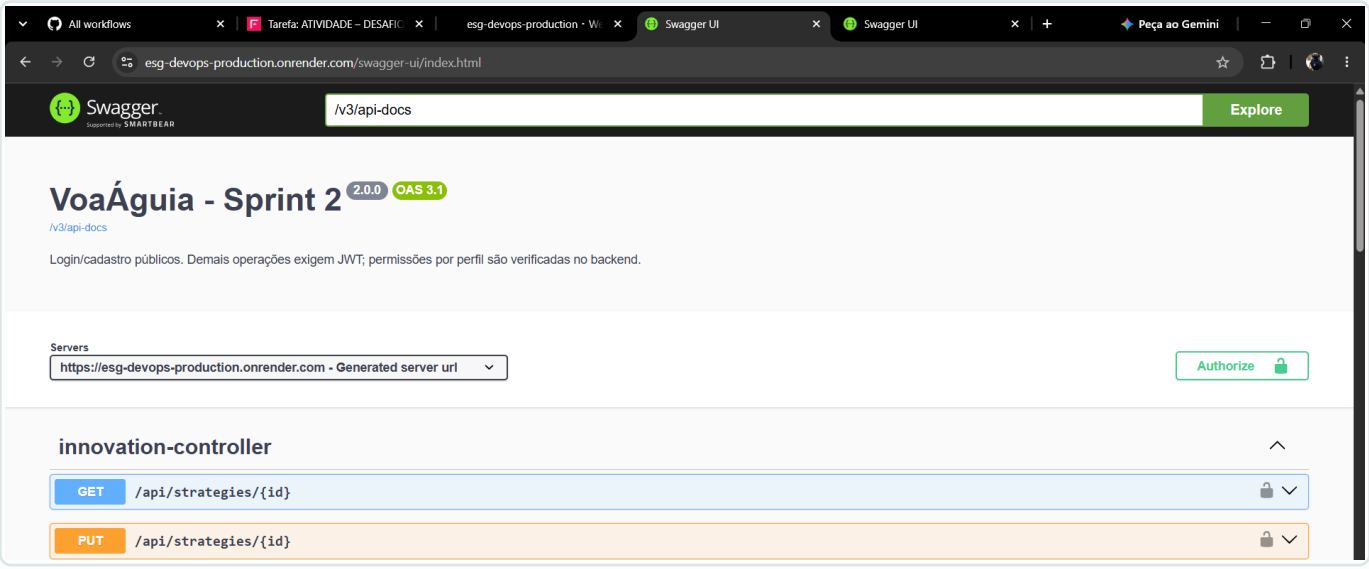


Figura 8. Swagger Voa Águia em URL HTTPS pública de produção acadêmica.

Persistência com MongoDB Atlas

Separação de bancos no mesmo cluster gratuito, com serviços de aplicação independentes.

Evidência 07 — Bancos para os dois ambientes

voaaguia_production	12.29 kB	0 B	3	6
voaaguia_staging	12.29 kB	0 B	3	6

Figura 9. Data Explorer do Atlas com os bancos voaaguia_production e voaaguia_staging.

Mapeamento da persistência

AMBIENTE	SERVIÇO HTTPS	BANCO MONGODB
Staging	esg-devops-staging.onrender.com	voaaguia_staging
Produção acadêmica	esg-devops-production.onrender.com	voaaguia_production

Isolamento e verificação

Cada Web Service utiliza uma `MONGODB_URI` configurada no próprio Render, direcionada ao seu banco. A recomendação de segurança é manter usuários e permissões `readWrite` restritos ao respectivo banco, senhas diferentes e lista de acesso de rede limitada aos IPs autorizados do Render.

Staging → homologação

Permite validar alterações antes de publicá-las na branch main.

Produção → entrega

Ambiente distinto, acessível publicamente para demonstração acadêmica.

Nota sobre evidência: a imagem do Atlas comprova a existência dos dois bancos, mas não exibe credenciais ou regras de rede — dados intencionalmente omitidos por segurança.

Desafios e soluções aplicadas

Problemas reais observados durante a implementação e medidas adotadas.

01 GitHub Actions sem RENDER_API_KEY

A execução interrompia o deploy por ausência de chave. Solução: cadastrar a API Key como Repository Secret e expô-la ao job.

02 MongoDB: Database name must not be empty

A inicialização do Spring Boot falhou sem um banco definido na URI. Solução: acrescentar o nome do banco à MONGODB_URI por ambiente.

03 HTTP 401 na rota /healthz

A verificação de saúde era bloqueada pelo Spring Security. Solução: tornar somente o health check público, sem liberar endpoints protegidos.

04 Separação staging versus produção

Configuração de dois Web Services Render, cada qual com branch, variáveis, JWT e banco MongoDB próprios.

05 Evitar deploy de código não validado

O workflow só solicita deploy após o job CI aprovar; o script publica o SHA específico e exige LIVE + teste /healthz.

Limitações: o plano Render Free pode suspender o serviço por inatividade; o MongoDB Atlas Free possui cotas de armazenamento e desempenho; a auditoria em H2 em memória não é persistente. A infraestrutura é adequada à demonstração acadêmica, não a um serviço comercial de alta disponibilidade.

Checklist obrigatório

Verificação final alinhada aos artefatos exigidos no enunciado da atividade.

Os itens concluídos têm base nos arquivos do pacote de entrega e nas capturas fornecidas. O ZIP foi montado a partir do código disponibilizado nesta conversa e deve ser conferido com a versão mais recente do GitHub antes do envio.

OK	Projeto compactado em .ZIP com estrutura organizada	ZIP de entrega preparado sem .env, target ou credenciais.
OK	Dockerfile funcional	Multiestágio Maven + Java 17; execução local comprovada.
OK	docker-compose.yml ou arquivos Kubernetes	API, MongoDB, rede e volume persistente definidos.
OK	Pipeline com etapas de build, teste e deploy	Jobs e deploys concluídos no GitHub Actions.
OK	README.md com instruções e prints	README atualizado e evidências vinculadas no pacote de entrega.
OK	Documentação técnica com evidências (PDF)	Este relatório inclui capturas reais das etapas principais.
OK	Deploy realizado em staging e produção	Ambos com log LIVE, health check OK e Swagger público.

Antes de enviar à plataforma: confira se o ZIP corresponde ao último commit aprovado do GitHub. Ele inclui `src/`, Dockerfile, Docker Compose, workflow, README, `.env.example`, as evidências e este PDF. O arquivo real `.env` não foi incluído por segurança.

Conclusão. O projeto Voa Águia demonstrou um fluxo funcional de CI/CD, com validação automática, containerização e deploys reais nos ambientes staging e produção acadêmica. As evidências confirmam a publicação e a verificação de saúde dos dois serviços.